
Reward-Conditioned Branch-Free Adaptive Weight 设计说明

面向 DiffusionNFT single-branch pseudo-target objective

从原版双分支归一化出发，构造一个只需要单个 reward-conditioned sample weight 的近似形式

对应推导	assets/ours_idea.tex
设计版本	Reward-conditioned root-second-moment normalization
日期	2026 年 7 月 27 日

目录

1 核心结论	1
2 问题背景	1
2.1 符号与 pseudo-target	1
2.2 原版 adaptive weighting 做了什么	2
3 为什么直接 post-merge normalization 不够理想	2
3.1 问题一：拟合 target 时分母必然触发下限	2
3.2 问题二：在 old-policy 点附近会弱化 reward 强度	3
4 新 weight 的推导	3
4.1 设计目标	3
4.2 从方向残差的二阶矩出发	3
4.3 为什么使用 $\frac{1}{D} \sum_j \sqrt{q_{i,j}}$	4
4.4 为什么必须 stop-gradient	4
5 三个关键性质	4
5.1 性质一：两个 reward 端点与原版严格一致	4
5.2 性质二：在 policy refresh 点与原版梯度严格一致	5
5.3 性质三：中间 reward 拟合 target 时 factor 不会普遍塌缩	5
6 如何理解 reward 的作用	6
6.1 均值项与方差项	6
6.2 为什么 refresh 点的 factor 暂时不依赖 reward	6
6.3 直接使用 normalized advantage 的形式	6
7 与另外两个版本的对比	7
8 梯度与优化含义	7
9 PyTorch 实现	7
9.1 实现注意事项	8
10 局限性与实验预期	9
10.1 它不在哪些方面等价	9
10.2 为什么预期优于直接简易版	9
10.3 推荐消融	9
11 总结	10

1 核心结论

新的 adaptive weight 不是把 single-branch loss 自身的误差直接放到分母，而是根据 reward、old-to-clean 残差和 current-to-old 更新量，估计一个 **reward-conditioned residual second moment**。对第 i 个样本和第 j 个维度，定义

$$d_{i,j} = (\mathbf{x}_{0,i}^{\text{old}} - \mathbf{x}_{0,i})_j, \quad \Delta_{i,j} = (\mathbf{x}_{0,i}^\theta - \mathbf{x}_{0,i}^{\text{old}})_j, \quad s_i = 2r_i - 1. \quad (1)$$

新的逐元素尺度为

$$q_{i,j} = (d_{i,j} + \beta s_i \Delta_{i,j})^2 + 4\beta^2 r_i (1 - r_i) \Delta_{i,j}^2 \quad (2)$$

并构造一个 detached 的 sample-level normalization factor :

$$w_i^{\text{RC}}(t) = \text{sg} \left[\max \left(\frac{1}{D} \sum_{j=1}^D \sqrt{q_{i,j}}, \epsilon \right) \right]. \quad (3)$$

最终的 single-branch objective 是

$$\ell_{i,\text{RC}} = \frac{\beta}{w_i^{\text{RC}}(t)} \left\| \mathbf{x}_{0,i}^\theta - \mathbf{x}_{0,i}^{\text{NFT}} \right\|^2. \quad (4)$$

这里需要区分两个容易混淆的概念：

- w_i^{RC} 是放在分母中的 **normalization factor**；
- 真正乘在该样本 MSE 前面的 **effective sample weight** 是 $\kappa_i^{\text{RC}} = \beta/w_i^{\text{RC}}$ 。

这个设计的核心价值是：它只需要一个 sample weight，不构造正负预测，但仍保留原版 DiffusionNFT adaptive factor 最重要的残差归一化性质。它在 $r_i = 0$ 和 $r_i = 1$ 两个端点与原版严格一致，在 $\mathbf{x}_{0,i}^\theta = \mathbf{x}_{0,i}^{\text{old}}$ 的 policy refresh 点与原版具有严格相同的局部梯度；对于中间 reward，它是一个有意设计的稳定近似。

2 问题背景

2.1 符号与 pseudo-target

DiffusionNFT 的 single-branch pseudo-target 可以写成

$$\mathbf{x}_{0,i}^{\text{NFT}} = \mathbf{x}_{0,i}^{\text{old}} + \frac{2r_i - 1}{\beta} (\mathbf{x}_{0,i} - \mathbf{x}_{0,i}^{\text{old}}). \quad (5)$$

使用

$$d_i = \mathbf{x}_{0,i}^{\text{old}} - \mathbf{x}_{0,i}, \quad \Delta_i = \mathbf{x}_{0,i}^\theta - \mathbf{x}_{0,i}^{\text{old}}, \quad s_i = 2r_i - 1, \quad (6)$$

可将其改写为

$$\mathbf{x}_{0,i}^{\text{NFT}} = \mathbf{x}_{0,i}^{\text{old}} - \frac{s_i}{\beta} d_i, \quad \mathbf{x}_{0,i}^\theta - \mathbf{x}_{0,i}^{\text{NFT}} = \Delta_i + \frac{s_i}{\beta} d_i. \quad (7)$$

$s_i \in [-1, 1]$ 是 centered reward：

- $s_i > 0$: pseudo-target 从 old prediction 向 clean sample 移动；
- $s_i < 0$: pseudo-target 向相反方向移动；

- $s_i = 0$: pseudo-target 等于 old prediction。

本文统一使用

$$\|\mathbf{u}\|^2 = \frac{1}{D} \sum_{j=1}^D u_j^2, \quad m(\mathbf{u}) = \frac{1}{D} \sum_{j=1}^D |u_j|. \quad (8)$$

2.2 原版 adaptive weighting 做了什么

用 d_i 和 Δ_i 表示原版的两个残差：

$$e_i^+ = d_i + \beta \Delta_i, \quad e_i^- = d_i - \beta \Delta_i. \quad (9)$$

对应的 detached normalization factors 是

$$w_i^+ = \text{sg}[\max(m(e_i^+), \epsilon)], \quad w_i^- = \text{sg}[\max(m(e_i^-), \epsilon)]. \quad (10)$$

原版 adaptive objective 为

$$\ell_{i,\text{sep}} = \frac{1}{\beta} \left[\frac{r_i}{w_i^+} \|e_i^+\|^2 + \frac{1-r_i}{w_i^-} \|e_i^-\|^2 \right]. \quad (11)$$

这两个分母有两个作用。第一，它们将较大的重建误差除以较大的尺度，避免困难样本或困难 timestep 完全主导梯度。第二，当 $w_i^+ \neq w_i^-$ 时，它们会改变两个方向的相对贡献，因此不仅改变梯度大小，还会改变严格合并后 pseudo-target 的位置。

如果要求与式 (11) 对任意中间 reward 严格等价，就必须保留 w_i^+ 、 w_i^- 的信息。严格合并后的梯度系数包含

$$\beta \left(\frac{r_i}{w_i^+} + \frac{1-r_i}{w_i^-} \right), \quad (12)$$

而 effective reward 还会变成

$$r_i^{\text{eff}} = \frac{r_i/w_i^+}{r_i/w_i^+ + (1-r_i)/w_i^-}. \quad (13)$$

因此，严格等价版不可避免地较复杂。

3 为什么直接 post-merge normalization 不够理想

当前简易版直接使用 single-branch residual：

$$w_i^{\text{post}} = \text{sg} \left[\max \left(m(\mathbf{x}_{0,i}^\theta - \mathbf{x}_{0,i}^{\text{NFT}}), \epsilon \right) \right], \quad \ell_{i,\text{post}} = \frac{\beta}{w_i^{\text{post}}} \|\mathbf{x}_{0,i}^\theta - \mathbf{x}_{0,i}^{\text{NFT}}\|^2. \quad (14)$$

它实现简单，但分母测量的是“当前模型距离优化目标还有多远”，而原版分母测量的是“reward 所作用的重建残差有多大”。这两者并不是同一个量。

3.1 问题一：拟合 target 时分母必然触发下限

当 $\mathbf{x}_{0,i}^\theta \rightarrow \mathbf{x}_{0,i}^{\text{NFT}}$ 时，

$$m(\mathbf{x}_{0,i}^\theta - \mathbf{x}_{0,i}^{\text{NFT}}) \rightarrow 0. \quad (15)$$

因此，未截断的分母必然趋近于零，实际实现只能依赖 ϵ 。虽然 ϵ 避免了数值除零，但分母会从 residual-dependent normalization 退化为由人工常数控制的区域。

尤其当 $r_i = 1/2$ 时， $s_i = 0$ 且 $\mathbf{x}_{0,i}^{\text{NFT}} = \mathbf{x}_{0,i}^{\text{old}}$ 。训练通常从 current model 接近 old model 的位置开始，因此一个几乎中性的样本很容易立即落入分母下限区域，即使 old prediction 与 clean sample 之间仍存在很大误差。

3.2 问题二：在 old-policy 点附近会弱化 reward 强度

在 $\mathbf{x}_{0,i}^\theta = \mathbf{x}_{0,i}^{\text{old}}$ ，即 $\Delta_i = 0$ 时，式 (7) 给出

$$w_i^{\text{post}} = \frac{|s_i|}{\beta} m(d_i) \quad (\text{忽略 } \epsilon). \quad (16)$$

因为 w_i^{post} 被 detach，其梯度为

$$\nabla_{\mathbf{x}_{0,i}^\theta} \ell_{i,\text{post}} \Big|_{\Delta_i=0} = \frac{2\beta}{D w_i^{\text{post}}} \left(\frac{s_i}{\beta} d_i \right) \quad (17)$$

$$= \frac{2\beta \text{sign}(s_i)}{D m(d_i)} d_i, \quad s_i \neq 0. \quad (18)$$

可以看到， $|s_i|$ 被分母抵消了。除 $s_i = 0$ 的精确点外，一个略高于平均的样本和一个 reward 极高的样本，可能获得近似相同的初始归一化梯度大小。reward 主要只剩下正负号，而不是连续的置信强度。

这与原版 DiffusionNFT 在 policy refresh 点的行为不同。原版此时 $w_i^+ = w_i^- = m(d_i)$ ，梯度连续地正比于 s_i 。新的设计首先要恢复这一性质。

4 新 weight 的推导

4.1 设计目标

我们希望一个新的 normalization factor 同时满足：

1. 每个样本只计算一个 detached scalar weight；
2. weight 显式依赖 reward，但不生成正负 branch prediction；
3. 在 current model 等于 old model 时，恢复原版的局部梯度；
4. 在 reward 两个端点恢复原版目标；
5. 对中间 reward，不因 signed residual 的抵消而异常变小；
6. 保留原版使用 mean absolute residual 进行尺度归一化的习惯。

4.2 从方向残差的二阶矩出发

为了说明新公式与原版的关系，可以引入一个只用于推导的方向变量 $Z_i \in \{-1, +1\}$ ：

$$\Pr(Z_i = +1) = r_i, \quad \Pr(Z_i = -1) = 1 - r_i. \quad (19)$$

它的均值和方差为

$$\mathbb{E}[Z_i] = 2r_i - 1 = s_i, \quad \text{Var}(Z_i) = 1 - s_i^2 = 4r_i(1 - r_i). \quad (20)$$

考虑第 j 个维度上的方向残差

$$E_{i,j}(Z_i) = d_{i,j} + \beta Z_i \Delta_{i,j}. \quad (21)$$

它的均值是

$$\mathbb{E}[E_{i,j}] = d_{i,j} + \beta s_i \Delta_{i,j}, \quad (22)$$

方差是

$$\text{Var}(E_{i,j}) = \beta^2 \text{Var}(Z_i) \Delta_{i,j}^2 = 4\beta^2 r_i(1 - r_i) \Delta_{i,j}^2. \quad (23)$$

由“二阶矩等于均值平方加方差”得到

$$\mathbb{E}[E_{i,j}^2] = (d_{i,j} + \beta s_i \Delta_{i,j})^2 + 4\beta^2 r_i (1 - r_i) \Delta_{i,j}^2 \quad (24)$$

$$= q_{i,j}. \quad (25)$$

这就是式 (2)。推导借助了原版的方向分布来解释它为什么合理，但实际计算只使用 d_i 、 Δ_i 和 r_i 三个量，不构造任何 branch prediction，也不维护 branch-specific weight。

4.3 为什么使用 $\frac{1}{D} \sum_j \sqrt{q_{i,j}}$

$\sqrt{q_{i,j}}$ 是第 j 个维度 residual 的 root second moment。先逐元素开方再取平均，是为了尽量保留原版 mean absolute error 的尺度：

- 当 reward 位于端点时， $\sqrt{q_{i,j}}$ 会严格退化为对应 residual 的绝对值；
- 因此 $\frac{1}{D} \sum_j \sqrt{q_{i,j}}$ 会严格退化为原版的 $m(e_i^\pm)$ ；
- 如果改为 $\sqrt{\frac{1}{D} \sum_j q_{i,j}}$ ，得到的是 global RMS，在 reward 端点也无法与原版 MAE factor 严格一致。

4.4 为什么必须 stop-gradient

w_i^{RC} 的目标是度量当前样本的更新尺度，而不是成为模型可以主动优化的对象。因此必须使用 `sg[]` 或在代码中置于 `torch.no_grad()` 上下文中。

如果不 detach，梯度会穿过分母。模型可能通过改变 $q_{i,j}$ 来改变自己的 loss weight，而不只是拟合 pseudo-target。这样不仅破坏与原版 adaptive factor 的对应关系，还可能诱导模型增大某些 residual 以减小 effective sample weight。

5 三个关键性质

5.1 性质一：两个 reward 端点与原版严格一致

当 $r_i = 1$ 时， $s_i = 1$ ，方差项为零：

$$q_{i,j} = (d_{i,j} + \beta \Delta_{i,j})^2, \quad w_i^{\text{RC}} = w_i^+. \quad (26)$$

同时

$$\mathbf{x}_{0,i}^\theta - \mathbf{x}_{0,i}^{\text{NFT}} = \Delta_i + \frac{1}{\beta} d_i. \quad (27)$$

所以

$$\ell_{i,\text{RC}} = \frac{\beta}{w_i^+} \left\| \Delta_i + \frac{d_i}{\beta} \right\|^2 = \frac{1}{\beta w_i^+} \|d_i + \beta \Delta_i\|^2, \quad (28)$$

与式 (11) 在 $r_i = 1$ 时完全相同。

同理，当 $r_i = 0$ 时，

$$q_{i,j} = (d_{i,j} - \beta \Delta_{i,j})^2, \quad w_i^{\text{RC}} = w_i^-, \quad (29)$$

并且 $\ell_{i,\text{RC}}$ 与原版 $r_i = 0$ 的目标完全相同。

5.2 性质二：在 policy refresh 点与原版梯度严格一致

当 current model 与 old model 相同时， $\Delta_i = 0$ 。此时对任意 reward，

$$q_{i,j} = d_{i,j}^2, \quad w_i^{\text{RC}} = \text{sg}[\max(m(d_i), \epsilon)] = w_i^+ = w_i^-. \quad (30)$$

记公共 factor 为 w_i^0 。由式 (11) 得到原版梯度

$$\nabla_{\mathbf{x}_{0,i}^\theta} \ell_{i,\text{sep}} \Big|_{\Delta_i=0} = \frac{2}{D} \left[\frac{r_i}{w_i^0} d_i - \frac{1-r_i}{w_i^0} d_i \right] \quad (31)$$

$$= \frac{2s_i}{D w_i^0} d_i. \quad (32)$$

新 objective 的梯度为

$$\nabla_{\mathbf{x}_{0,i}^\theta} \ell_{i,\text{RC}} \Big|_{\Delta_i=0} = \frac{2\beta}{D w_i^0} \left(\frac{s_i}{\beta} d_i \right) \quad (33)$$

$$= \frac{2s_i}{D w_i^0} d_i. \quad (34)$$

式 (32) 与式 (34) 完全相同。

这条性质很重要，因为每次 old policy 更新后，current 与 old 通常接近。新 weight 在训练最常访问的局部区域保持了原版 adaptive objective 的 reward 响应：梯度大小连续地正比于 $s_i = 2r_i - 1$ 。它不会像 post-merge 版本那样把 $|s_i|$ 从初始梯度中归一化掉。

5.3 性质三：中间 reward 拟合 target 时 factor 不会普遍塌缩

当 current prediction 正好等于 pseudo-target 时，

$$\Delta_i^* = -\frac{s_i}{\beta} d_i. \quad (35)$$

代入式 (2)：

$$q_{i,j}^* = (1 - s_i^2)^2 d_{i,j}^2 + (1 - s_i^2) s_i^2 d_{i,j}^2 \quad (36)$$

$$= (1 - s_i^2) d_{i,j}^2. \quad (37)$$

因此在忽略 ϵ 时，

$$w_i^{\text{RC}}(\mathbf{x}_{0,i}^\theta = \mathbf{x}_{0,i}^{\text{NFT}}) = \sqrt{1 - s_i^2} m(d_i) = 2\sqrt{r_i(1 - r_i)} m(d_i). \quad (38)$$

对任何严格的中间 reward $0 < r_i < 1$ ，只要 old-to-clean error 不为零，这个 factor 就仍然为正。几个典型情况如下。

表 1: 拟合 pseudo-target 时的新 normalization factor，表中省略 $m(d_i)$ 和 ϵ 。

r_i	0	0.25	0.5	0.75	1
$s_i = 2r_i - 1$	-1	-0.5	0	0.5	1
$2\sqrt{r_i(1 - r_i)}$	0	0.866	1	0.866	0

特别地， $r_i = 0.5$ 时

$$w_i^{\text{RC}} = m(d_i), \quad (39)$$

而 post-merge factor 为零并触发 ϵ 。端点处新 factor 也会趋于零，但这是合理的：此时唯一被 reward 选择的 residual 已经被完全拟合，原版 factor 同样会触发 ϵ 。

6 如何理解 reward 的作用

6.1 均值项与方差项

式 (2) 可以分成两部分：

$$\underbrace{(d_{i,j} + \beta s_i \Delta_{i,j})^2}_{\text{reward-directed mean residual}} + \underbrace{4\beta^2 r_i (1 - r_i) \Delta_{i,j}^2}_{\text{directional variance}}. \quad (40)$$

均值项描述 reward 指定的净方向；方差项描述当 reward 不确定时，不能被一个 signed mean residual 表示的剩余尺度。

- 当 r_i 接近 0 或 1，reward 方向明确，方差项趋于零；
- 当 $r_i = 0.5$ ，方向最不确定，方差系数达到最大值；
- 方差项始终非负，所以即使均值项发生向量抵消，也不会轻易得到一个不合理的小 factor。

6.2 为什么 refresh 点的 factor 暂时不依赖 reward

在 $\Delta_i = 0$ 时，式 (30) 对所有 reward 都等于 $m(d_i)$ 。这不是缺少 reward conditioning，而是有意保留原版的局部几何结构。

此时 reward 已经通过 pseudo-target 控制梯度：

$$\nabla_{\mathbf{x}_{0,i}^\theta} \ell_{i,\text{RC}} \propto s_i d_i. \quad (41)$$

normalization factor 只负责消除不同样本 d_i 尺度的差异，不应再把 $|s_i|$ 消掉。当 current model 离开 old model 后， $\Delta_i \neq 0$ ， w_i^{RC} 会通过均值项和方差项显式依赖 reward。

6.3 直接使用 normalized advantage 的形式

如果后续方法使用归一化 reward 或 advantage $\hat{A}_i \in [-1, 1]$ ，可以直接令

$$\hat{A}_i = s_i = 2r_i - 1. \quad (42)$$

此时新 factor 可以写成更适合 single-branch 方法的形式：

$$q_{i,j} = \left(d_{i,j} + \beta \hat{A}_i \Delta_{i,j} \right)^2 + \beta^2 \left(1 - \hat{A}_i^2 \right) \Delta_{i,j}^2. \quad (43)$$

这个表达式完全不需要正负 branch 的语义，只需要：

- old prediction error d_i ；
- current policy displacement Δ_i ；
- 每个样本的 normalized reward $\hat{A}_i$ 。

如果使用论文中的线性 sample weight $W_i = 1 + \hat{A}_i \in [0, 2]$ ，也可以直接使用 $\hat{A}_i = W_i - 1$ 代入式 (43)。因此，这个 adaptive factor 可以自然接到后续“每个样本只有一个 reward weight”的推导中。

表 2: 三种 adaptive weighting 形式的主要差异。

	严格等价版	直接 post-merge 简易版	新 reward-conditioned 版
Normalization factors	两个: w_i^+, w_i^-	一个: w_i^{post}	一个: w_i^{RC}
是否构造方向预测	是	否	否
中间 reward 严格等价	是	否	否
是否保留 adaptive target shift	是	否	否
refresh 点局部梯度	原版基准	一般只保留 reward 符号	与原版严格一致
拟合 pseudo-target 时的分母	由两个 residual 决定	普遍触发 ϵ	中间 reward 通常保持非零
reward 如何进入 factor	通过两个 factor 的混合	通过 pseudo-target residual	直接进入均值与方差
计算与实现复杂度	最高	最低	介于两者之间，但仍是单 weight

7 与另外两个版本的对比

严格等价版适合作为理论基准。post-merge 版最简单，但它会把 pseudo-target 距离和 adaptive normalization 混为同一个量。新版本的定位是：在不恢复 branch-specific target shift 的前提下，尽可能保留原版的 residual-scale normalization、reward 连续性和关键边界行为。

8 梯度与优化含义

由于 w_i^{RC} 被 detach，新 loss 对 current prediction 的梯度非常清晰：

$$\nabla_{\mathbf{x}_{0,i}^\theta} \ell_{i,\text{RC}} = \frac{2\beta}{D w_i^{\text{RC}}} \left(\mathbf{x}_{0,i}^\theta - \mathbf{x}_{0,i}^{\text{NFT}} \right). \quad (44)$$

因此：

- 梯度方向完全由 single-branch pseudo-target 决定；
- 梯度大小由单个 sample multiplier $\kappa_i^{\text{RC}} = \beta/w_i^{\text{RC}}$ 调整；
- adaptive factor 不会像严格等价版那样再次移动 pseudo-target；
- 这一职责分离更符合后续方法的概念结构：归一化 reward 决定样本 目标或样本权重；adaptive factor 只负责控制该样本的更新尺度。

对于 residual 很大的样本， w_i^{RC} 通常也较大，因此更新被压缩；对于 residual 较小但未进入 ϵ 区域的样本，更新相对增强。这延续了原版 DiffusionNFT 的“按样本和 timestep 归一化优化难度”的思想。

9 PyTorch 实现

下面给出与公式直接对应的实现框架。关键要求是：用于 factor 的 `x0_prediction` 必须在计算 q_i 时 detach，但用于 MSE 的 `x0_prediction` 必须保留梯度。

```

# x0_prediction: current model clean prediction, with gradients
# x0_old_prediction, x0, reward: detached / no-grad tensors
x0_prediction = xt - t_expanded * forward_prediction
x0_old_prediction = xt - t_expanded * old_prediction.detach()

reduce_dims = tuple(range(1, x0.ndim))
r_view = r.view(-1, *([1] * (x0.ndim - 1)))

with torch.no_grad():
    d = (x0_old_prediction - x0).double()
    delta = (x0_prediction - x0_old_prediction).double()
    r64 = r_view.double()
    s = 2.0 * r64 - 1.0

    q = (
        (d + config.beta * s * delta).square()
        + 4.0
        * (config.beta ** 2)
        * r64
        * (1.0 - r64)
        * delta.square()
    )

    # Guard against a tiny negative value caused by finite precision.
    weight_factor = (
        q.clamp_min(0.0)
        .sqrt()
        .mean(dim=reduce_dims, keepdim=True)
        .clamp_min(1e-5)
        .to(x0_prediction.dtype)
    )

    x0_target = (
        x0_old_prediction
        + (s / config.beta).to(x0.dtype)
        * (x0 - x0_old_prediction)
    )

sample_loss = (
    config.beta
    * (x0_prediction - x0_target).square()
    / weight_factor
).mean(dim=reduce_dims)

policy_loss = sample_loss.mean()

```

9.1 实现注意事项

1. **Reward 范围。** 公式要求 $r_i \in [0, 1]$ ，等价地要求 $s_i \in [-1, 1]$ 。如果输入是未经限制的 advantage，应先标准化和裁剪。否则 $4r_i(1 - r_i)$ 或 $1 - s_i^2$ 可能为负。
2. **精度。** 建议像原版 factor 一样使用 float64 计算 q_i 和均值，再转回模型 dtype。特别是在 mixed precision 下，这能减少平方和开方带来的舍入问题。
3. **非负保护。** 理论上 $q_{i,j} \geq 0$ ，因为它是两个平方项之和。代码中的 `clamp_min(0)` 只用于防止有限精度产生极小负值。

4. **下限 ϵ 。** 应与原版使用同一个 ϵ ，例如 10^{-5} ，这样端点行为和 loss scale 更可比。若改变 ϵ ，需同时观察 floor activation ratio。
5. **按 **sample** 归约。** w_i^{RC} 应对通道和空间维度取平均，但保留 batch 维。不要先对整个 batch 求平均，否则会失去 sample-wise normalization。
6. **日志。** 建议记录 w_i^{RC} 、 β/w_i^{RC} 、触发 ϵ 的样本比例，以及按 reward 和 timestep 分桶后的均值。

10 局限性与实验预期

10.1 它不在哪些方面等价

对 $0 < r_i < 1$ ，新版本并不严格等价于原版。原因是原版的两个 detached factor 会同时产生：

- 一个由 $r_i/w_i^+ + (1 - r_i)/w_i^-$ 决定的 harmonic-like gradient scale；
- 一个由 w_i^+ 与 w_i^- 的差异引起的 adaptive pseudo-target shift。

单个 second-moment factor 无法同时保留这两个自由度。新设计主动放弃 adaptive target shift，只保留原始 single-branch pseudo-target，并用一个 reward-conditioned scalar 控制更新大小。

10.2 为什么预期优于直接简易版

预期改善来自以下三点，但最终仍需实验验证：

1. 在 policy refresh 点恢复与原版相同的 reward-linear gradient，不会把 reward magnitude 归一化成近似符号；
2. 对中间 reward，即使 current prediction 拟合 pseudo-target，factor 仍保留 old-to-clean residual 的尺度；
3. 方差项阻止 signed mean residual 的偶然抵消，使 sample weight 随 reward 和 policy displacement 更平滑地变化。

新 factor 使用 root second moment。由 RMS-AM 关系，它可能比对应的 reward-weighted mean absolute residual 更保守，因此某些区间的 β/w_i^{RC} 可能偏小。这不是数值错误，而是该近似用稳定性换取简洁性的结果。若整体更新显著变小，应优先使用一个全局 loss-scale coefficient 统一校准，而不是破坏 q_i 的 reward 结构。

10.3 推荐消融

为了判断它是否确实优于 post-merge 简易版，建议在完全相同的 rollout、reward、 β 、学习率和随机种子下比较：

- exact adaptive single-branch；
- direct post-merge adaptive；
- reward-conditioned branch-free adaptive；
- 不使用 adaptive factor 的 single-branch baseline。

除最终 reward 外，建议同时观察：

- 训练前期不同 reward bin 的平均梯度范数；
- w_i 与 effective multiplier β/w_i 的分布；
- 分母触发 ϵ 的比例；
- 不同 timestep 的归一化尺度；
- current model 与 old model 的距离，以及训练是否出现突发梯度。

11 总结

新 adaptive weight 可以概括为以下思路：

不再分别问“正方向 residual 多大”和“负方向 residual 多大”，而是根据当前样本的 centered reward，直接估计 reward-directed residual 的二阶尺度，并将它作为该样本唯一的 detached normalization factor。

它比严格等价版更简单，因为只计算一个 scalar factor 且不改变 single-branch pseudo-target；它又比直接 post-merge 版更接近原版，因为它恢复了 reward 端点、policy refresh 点和 residual-scale normalization 的关键行为。

最简洁的 normalized-advantage 表达为

$$w_i^{\text{RC}} = \text{sg} \left[\max \left(\frac{1}{D} \sum_{j=1}^D \sqrt{\left(d_{i,j} + \beta \hat{A}_i \Delta_{i,j} \right)^2 + \beta^2 (1 - \hat{A}_i^2) \Delta_{i,j}^2}, \epsilon \right) \right], \quad (45)$$

其中 $\hat{A}_i \in [-1, 1]$ 。最终每个样本只有一个 effective weight β/w_i^{RC} ，这与后续基于 reward 设计 sample-wise weight 的方法 具有更一致的概念结构。